SpotCast

Project report

SWMAD-01 Mobile Application Development

Group members and study numbers: [FILL IN: name, study number, one line per member]

Delivery date: [FILL IN]

Test user: [FILL IN: e-mail] / [FILL IN: password]

The test user must exist in the Firebase project that the handed-in google-services.json points to.

1. App vision

People who spend time outdoors keep a mental list of places: a beach that works in a westerly wind, a trail head with parking, a viewpoint worth the climb. Before going they check the weather for each place by typing its name into a weather app, which often gives the forecast for the nearest town and not for the place itself.

SpotCast lets the user save a spot at the position where they are standing, with a name, a category and a note. A place the user has not been to yet can be saved too, by typing its address or its coordinates and confirming it on a map. The spots are stored in the user's account and shown in a list and on a map. Opening a spot shows the current weather and a seven-day forecast for its exact coordinates.

2. Features and requirements

User stories

US1. Save where I am. As a hiker, I want to save the place where I am standing with one button, so that I do not have to type coordinates or search for an address. *Accepted when:* the add screen asks for the location permission the first time, reads the position by itself, and the save button stays disabled until there is both a name and a position.

US2. Forecast for the spot itself. As a surfer, I want the forecast for the exact position of my spot, so that I can decide whether the trip is worth it. *Accepted when:* the details screen shows temperature, wind and weather now, and minimum, maximum and chance of rain for seven days, for the stored coordinates. If the call fails the screen says why and offers to try again, and the rest of the screen still works.

US3. Spots on a map. As a user with several spots, I want to see them on a map together with my own position, so that I can combine the ones that are close to each other. *Accepted when:* every spot is a marker, and tapping the marker's info window opens the details of that spot.

US4. Spots follow the account. As a user with two devices, I want a spot saved on one device to show up on the other without me refreshing. *Accepted when:* the list changes while it is open if a spot is added or deleted elsewhere, and a user never sees the spots of another user.

US5. Save a place before going there. As someone planning a trip, I want to save a spot by its address or its coordinates, so that I can follow its forecast before I have ever been there. *Accepted when:* the add screen offers "Another place" next to "Where I am now"; an address is looked up and typed coordinates are checked for valid ranges; the result is shown as a pin on a map under the question "Is this the right place?"; and the position is only used after the user answers yes.

US6. Light or dark. As a user who checks the app in the evening, I want to choose between a light and a dark theme, or let the app follow the phone, so that the screen is comfortable to read. *Accepted when:* the profile offers System, Light and Dark; the whole app changes at once; and the choice is still there after the app is restarted.

How the mandatory requirements are met

- **Kotlin and coroutines.** All service functions are suspend functions. View models start them in viewModelScope, and Firebase and location tasks are awaited with await().
- **Jetpack Compose.** All UI is composables with Material 3: Scaffold, top bar, bottom navigation bar, cards, lazy list.
- **Compose Navigation.** One NavHost with seven routes. The details route takes the spot

id as an argument. The back stack is cleared on sign in and sign out.

- **Four or more screens.** Spots, Add spot, Spot details, Map and Profile, besides Sign in and Sign up.
- **Firebase Authentication.** E-mail and password, behind the interface AuthService.
- **Firestore.** Spots are documents in users/{uid}/spots. Security rules limit each user to their own documents.
- **External HTTP API.** The Ktor client calls two APIs: the Open-Meteo forecast API for the weather, and Nominatim, the address search of OpenStreetMap, to turn a typed address into a position. Each answer is mapped to a sealed class with a success case and named failure cases.
- **Additional technology.** GPS through the fused location provider, and maps from OpenStreetMap shown with the osmdroid library. OpenStreetMap was chosen over Google Maps because it needs no billing account and no API key. The spots list also uses Firestore real-time updates.

3. UI and user flow

The sketch shows the seven screens. The app starts on Sign in, or directly on Spots if Firebase still has a signed-in user. Spots, Map and Profile are the three tabs of the bottom bar. Add spot and Spot details are opened on top of a tab and have a back arrow.

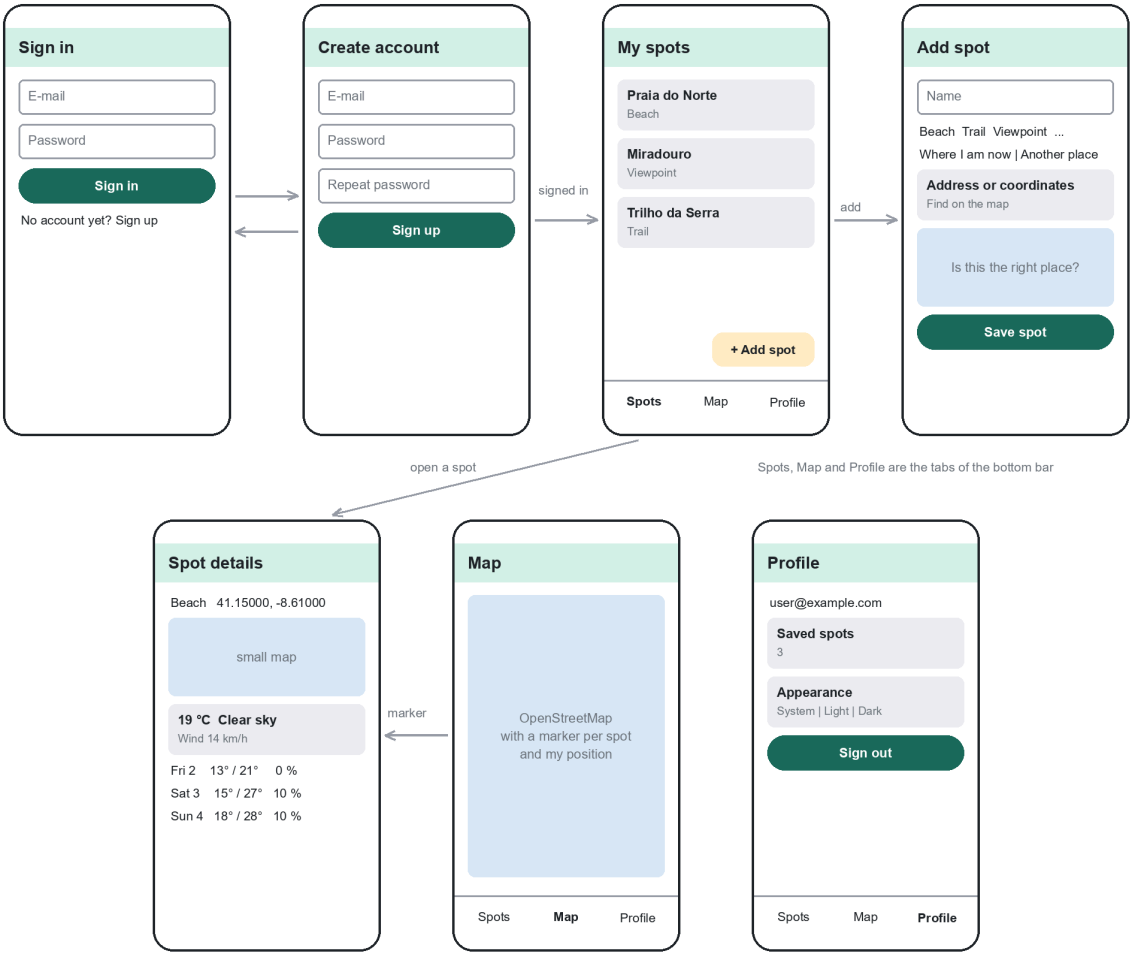
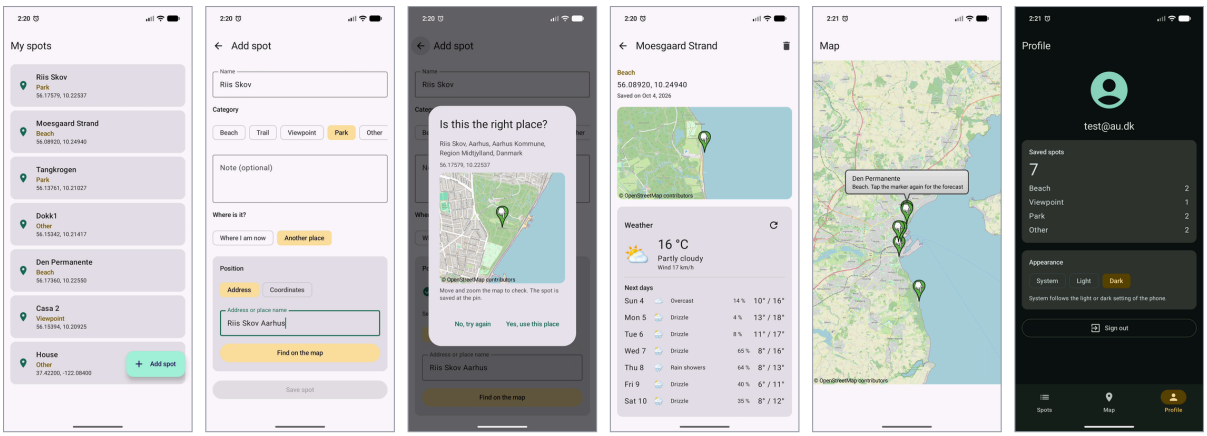


Figure 1. The screens and the navigation between them.



Spots | Add spot | Place check | Spot details | Map | Profile, dark theme

Figure 2. The app running in the Pixel 10a emulator. The last screen is the profile with the dark theme chosen.

4. Components

The code is split in three packages. **ui** holds the composables and one view model per group of screens. **domain** holds plain Kotlin models with no reference to Firebase or Ktor. **integration** holds one service per external component. Composables call view models, view models call service interfaces, and only the service implementations know about the external components. The one exception is the map. osmdroid is a classic Android view, so it is wrapped in one composable, OsmMap, which lives in the UI and fetches its own tiles.

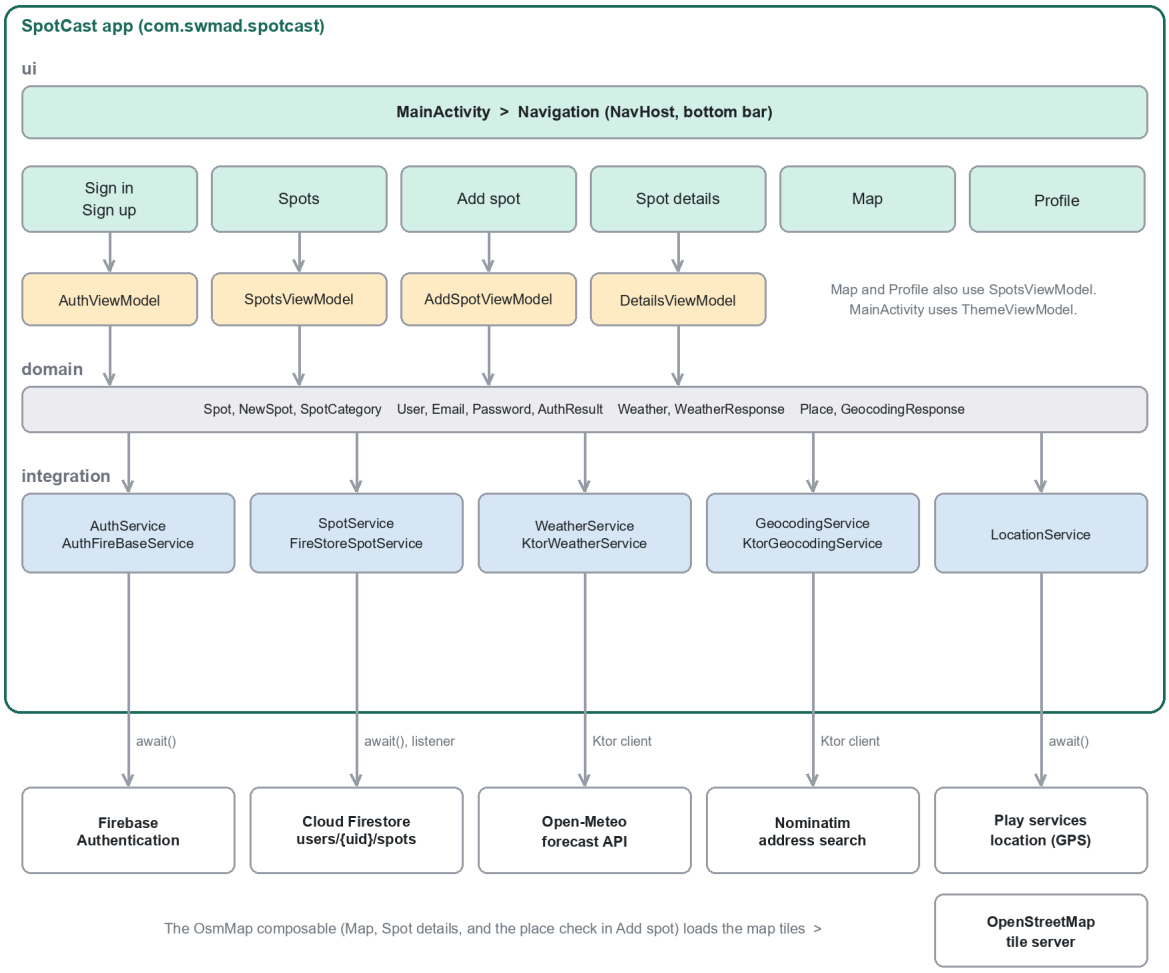


Figure 3. Components of the app and the external components they use.

Firestore data is read into SpotFS, a class with the @DocumentId annotation and default values, and then mapped to the domain class Spot, where the id cannot be null. The same split is used for the weather: the serializable classes mirror the JSON from Open-Meteo, and the screens only see the domain class Weather.

The theme is the one setting that belongs to the device and not to the account. It is held by ThemeViewModel, whose owner is the activity, and stored in SharedPreferences. MainActivity reads it and passes light or dark to the theme composable, so every screen follows without knowing about the setting.

5. Use cases

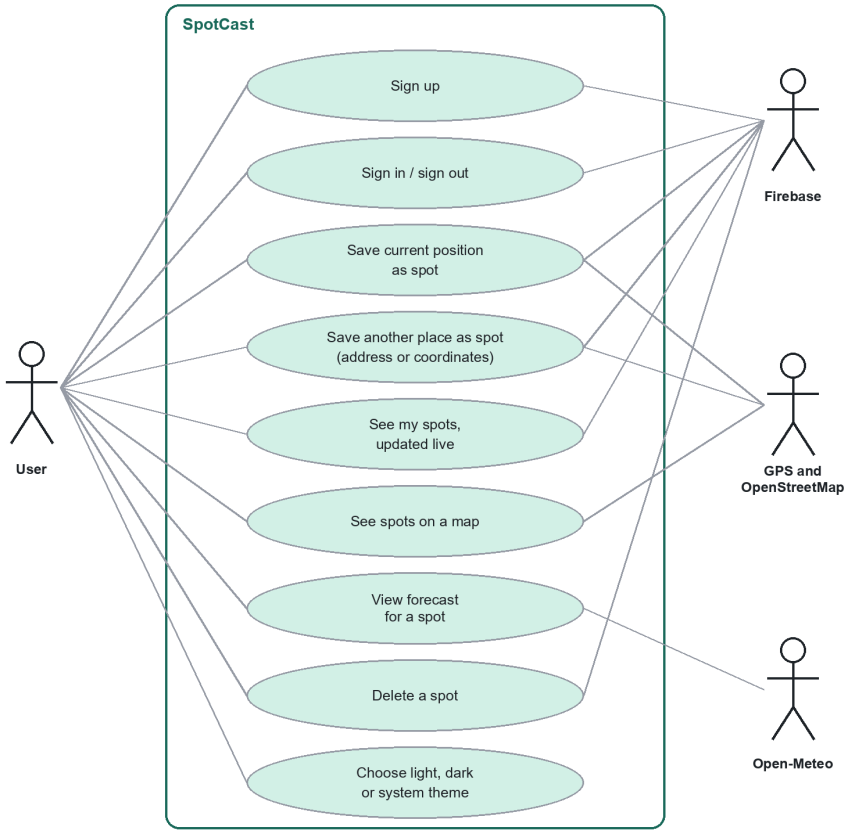


Figure 4. Use cases.

Save current position as spot. The user opens Add spot. If the location permission is missing, the system asks for it. When it is granted, the app reads the position and shows the coordinates. The user types a name, picks a category and saves. The app writes the document to Firestore and returns to the list, where the snapshot listener has already delivered the new spot. *Alternatives:* permission denied, the card explains why the permission is needed and offers to ask again; no position, an error is shown and the user can read the position again; the write fails, the error is shown and the form keeps its content.

Save another place as spot. In Add spot the user chooses "Another place" and types either an address or a latitude and longitude. For an address the app asks Nominatim for the best match; coordinates are checked to be numbers inside the valid ranges. The app then opens a dialog with the resolved address, the coordinates and a map with a pin, and asks "Is this the right place?". On yes the position is filled in and the spot can be saved as usual; on no the user is back in the form and can try again. *Alternatives:* nothing is found, no connection, or invalid coordinates: each gives its own message under the form.

View forecast for a spot. The user opens a spot from the list or from the map. The app loads the spot from Firestore, then calls Open-Meteo with its coordinates, and shows weather now and seven days ahead. *Alternatives:* no connection, the service is unavailable, or the position is rejected: each gives its own message in the weather card with a button to try again. The spot and the map are shown either way.

6. Conclusion

The app covers the mandatory requirements with a small number of screens that each have a clear job. The structure taken from the lessons, with an interface per service and a separate domain model, paid off when the weather was added: the details screen did not need to know anything about HTTP, only about the three states loading, loaded and error.

Two things took more thought than expected. The first was navigation around sign in and sign out. Navigating to the list after sign in is not enough, because back then returns to the sign-in screen, and after sign out the view models of the old user are still alive. The solution was to clear the whole back stack in both directions. The second was the Firestore listener: wrapped in a Flow and collected in viewModelScope, it is removed when the view model is cleared, so no listener leaks.

[FILL IN: what the group learned and what it would do differently, in your own words.]

7. Known bugs and issues

- [FILL IN with anything you find in your own testing. Run in the Pixel 10a emulator against the real Firebase project so far: adding a spot by GPS and by address, the live list, the forecast, the map with markers, the profile and the theme switch. Not yet run: sign up and sign in through the forms, deleting a spot, signing out, typed coordinates, and two devices at the same time.]
- The address search uses only the best match. If the wrong place comes up, the user has to answer no and type a more precise address; there is no list of matches to pick from.
- Saving a spot waits for the answer from Firestore. Without a connection the save button keeps spinning until the connection is back.
- The position is a single GPS reading. Indoors it can be off, and the app does not show the accuracy.
- A spot cannot be edited after it is saved, only deleted.
- The map itself stays light in the dark theme, because the map tiles from OpenStreetMap come in one style.
- The map moves to the newest spot the first time it opens and does not zoom to fit all spots.
- Texts are in English only and are written in the code, not in string resources.
- There are no automated tests.